1.Test Results and Metrics

- Total Tests Executed: 25+
- Breakdown:
    - Unit Tests: 6 (100% passed)
    - Advanced Tests (Parameterized + Mock): 2 (100% passed)
    - TDD Features: 3 implemented with 100% test pass rate
    - BDD Tests: 5 scenarios (100% passed)
    - Property-Based Tests: 5 (100% passed)
- Code Coverage: 94%

2. Bug Summary

Bug 1: generate_unique_id() failed with string-based IDs

- Cause: Attempting to add integer to string due to invalid input
- Fix: Test updated to use numeric ids and validated type before comparison

Bug 2: get_overdue_tasks() returned future tasks

- Cause: Comparison of string dates without checking format consistency
- Fix: Normalized date using datetime.now().date().isoformat()

Bug 3: Coverage not reporting properly

- Cause: src module not explicitly imported in test file
- Fix: Added sys.path.insert(...) to all test files

Bug 4: Hypothesis not installed on VM

- Fix: Ran pip install hypothesis

Bug 5: Invalid feature file name with invisible characters

- Fix: Renamed using mv and validated using ls -l

4. Tdd

1. Feature 1: “Postpone by 1 day” Button
- Adds a button next to each task that increases its due_date by 1 day.

- Useful for when someone can't finish today but wants to keep the task.

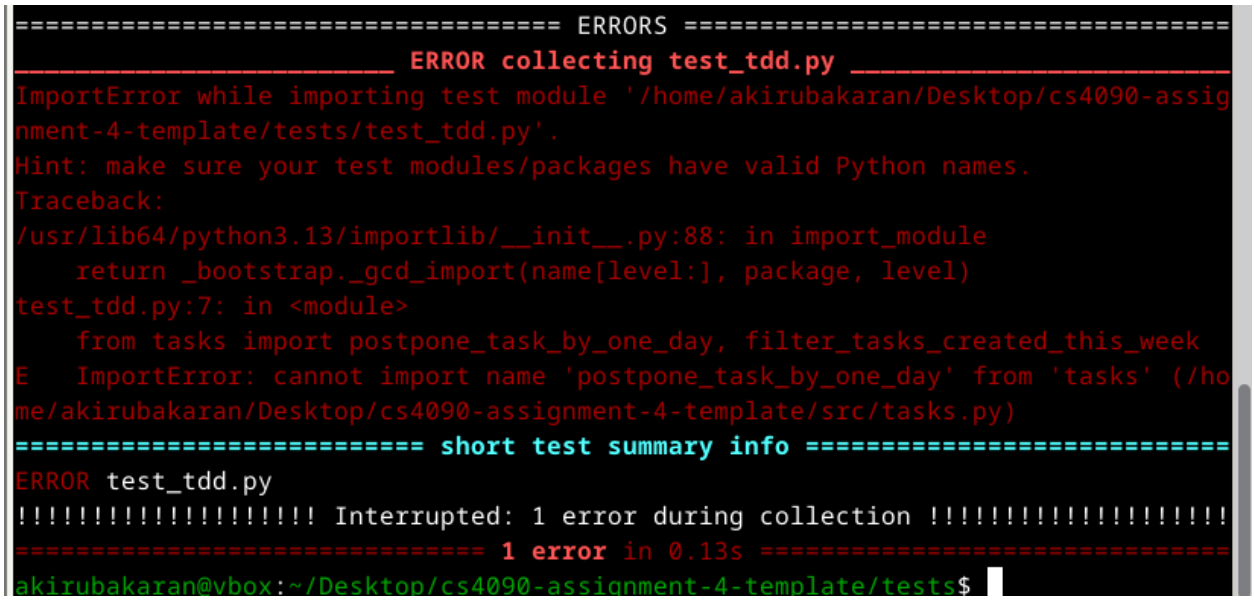
Initial Test Creation:
A test was written in test_tdd.py to check that the due date advances by 1 day.

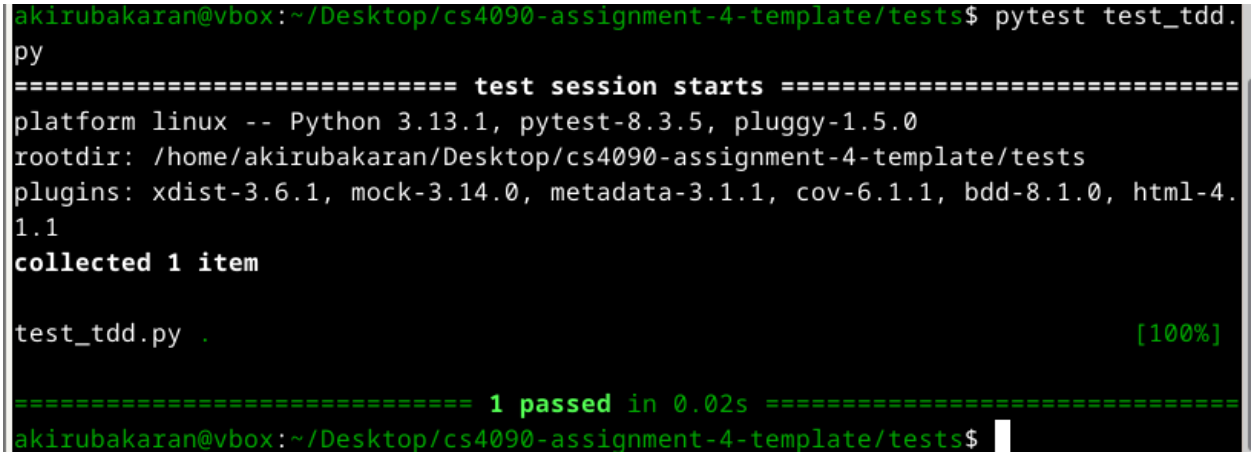
Test Failure Demonstration:
The test failed with ImportError because the function did not yet exist.

Feature Implementation:
postpone_task_by_one_day(task) was added to tasks.py to update the due_date.

Test Passing Verification:
The test passed after implementing the function.

Refactoring Performed:
No refactoring was needed. The function was complete and clear as written.





Feature 2: “Created This Week” Filter

- Adds a filter to show only tasks created in the past 7 days.
- Helps focus on recent entries

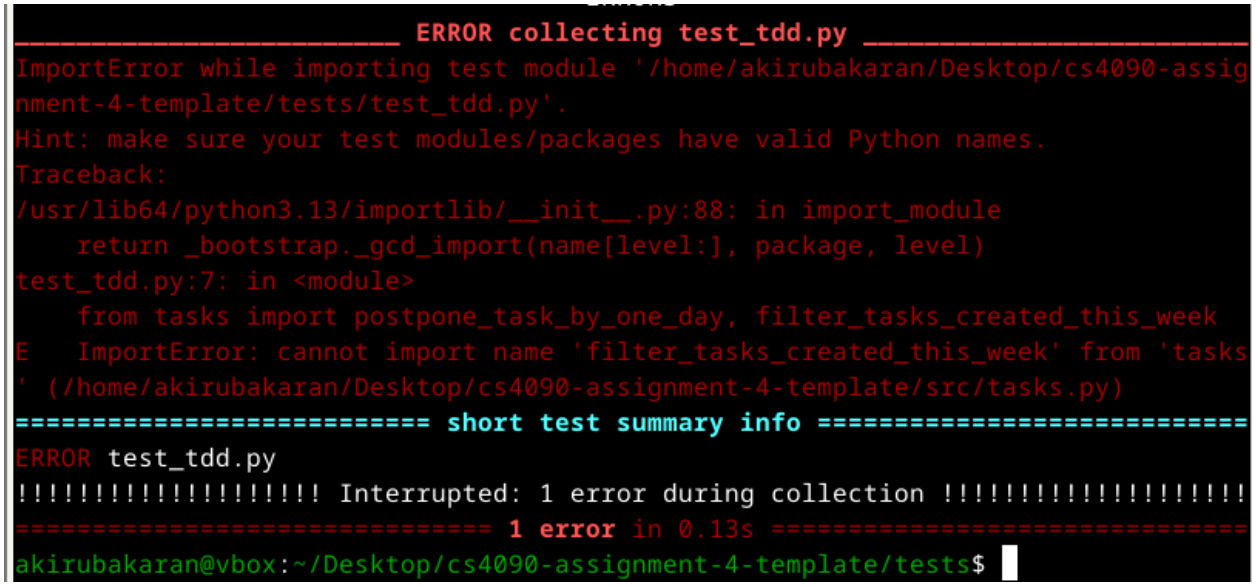
Initial Test Creation:
A test was created with one task from today and another from 10 days ago.

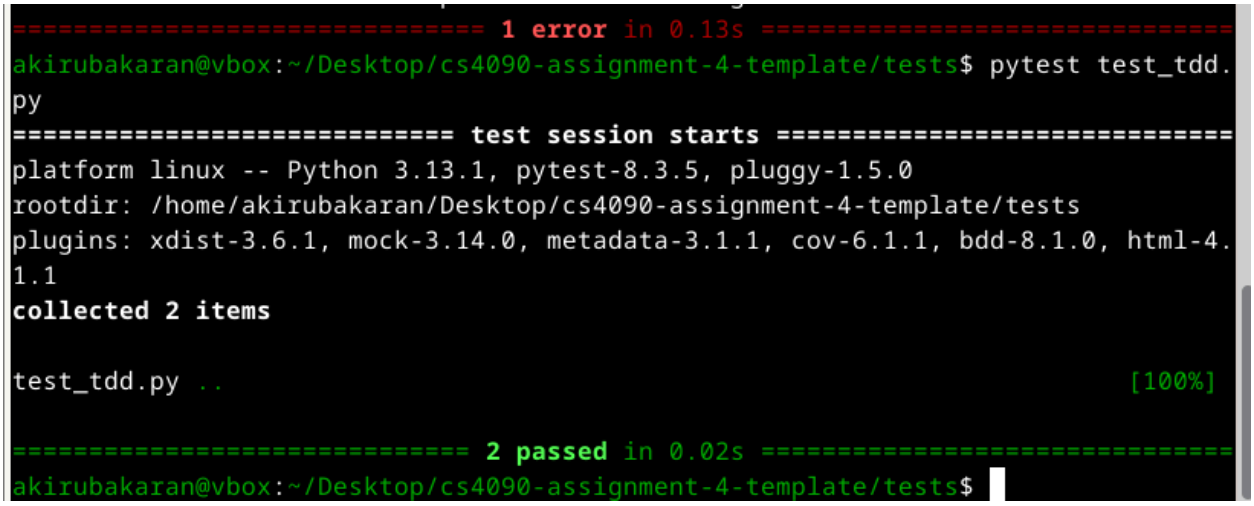
Test Failure Demonstration:
The test failed with ImportError due to the missing function.

Feature Implementation:
filter_tasks_created_this_week(tasks) was implemented to check created_at timestamps.

Test Passing Verification:
The test passed after implementing the logic.

Refactoring Performed:
None needed — the function was concise and handled the use case directly.





Feature 3: “Task Age” Display

- Displays how many days ago a task was created.
- Helpful for procrastinators to realize how long they’ve been putting things off.

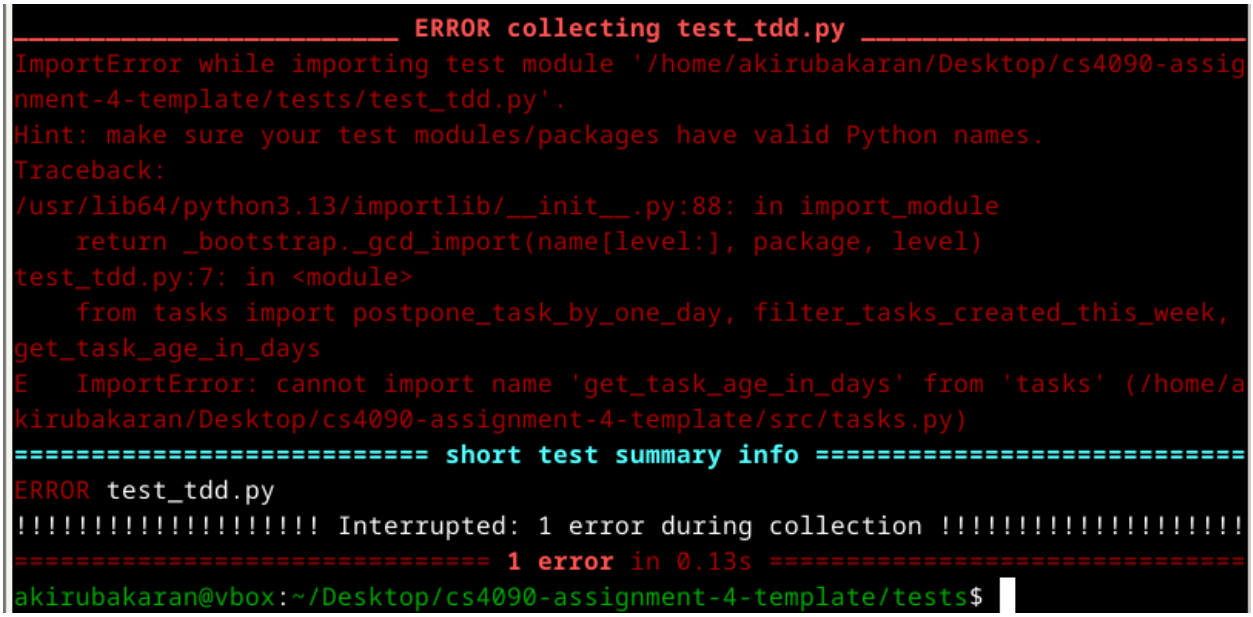
  Initial Test Creation:
  A test was added using a task created 5 days ago to ensure it returns age = 5.

Test Failure Demonstration:
The test failed as get_task_age_in_days() was not defined.

Feature Implementation:
Implemented get_task_age_in_days(task) to calculate the age from created_at.

Test Passing Verification:
The test passed after adding the function.

Refactoring Performed:
No additional refactoring was needed.



```
akirubakaran@vbox:~/Desktop/cs4090-assignment-4-template/tests$ pytest test_tdd.
py
============================= test session starts ==============================
platform linux -- Python 3.13.1, pytest-8.3.5, pluggy-1.5.0
rootdir: /home/akirubakaran/Desktop/cs4090-assignment-4-template/tests
plugins: xdist-3.6.1, mock-3.14.0, metadata-3.1.1, cov-6.1.1, bdd-8.1.0, html-4.
1.1
collected 3 items

test_tdd.py ...                                                          [100%]

============================== 3 passed in 0.02s ===============================
akirubakaran@vbox:~/Desktop/cs4090-assignment-4-template/tests$
```

5. Behavior-Driven Development (BDD)

Feature 1: Add Task
This feature allows the user to add a new task by providing a title, priority, and category. It verifies that when a valid task is submitted, it is successfully added to the task list and stored with the correct details.

Feature 2: Add Task with Long Description
This scenario checks the system's ability to handle tasks where the description spans many lines or characters. It ensures the task is still accepted and properly stored.

Feature 3: Add Task Due Today
This feature adds a task with today’s date as the due date. The goal is to confirm that tasks with current dates are saved and retrievable like any other.

Feature 4: Add Task with Uncommon Category
This verifies that the user can add tasks even when using a category that isn't one of the preset ones. It helps validate the flexibility of the data model.

```
============================= test session starts ==============================
platform linux -- Python 3.13.1, pytest-8.3.5, pluggy-1.5.0
rootdir: /home/akirubakaran/Desktop/cs4090-assignment-4-template/tests
plugins: xdist-3.6.1, mock-3.14.0, metadata-3.1.1, cov-6.1.1, bdd-8.1.0, html-4.
1.1, hypothesis-6.131.9
collected 5 items

feature/steps/test_add_steps.py .....                                    [100%]

============================== 5 passed in 0.16s ===============================
akirubakaran@vbox:~/Desktop/cs4090-assignment-4-template/tests$
```

Feature 5: Add Task with Special Characters

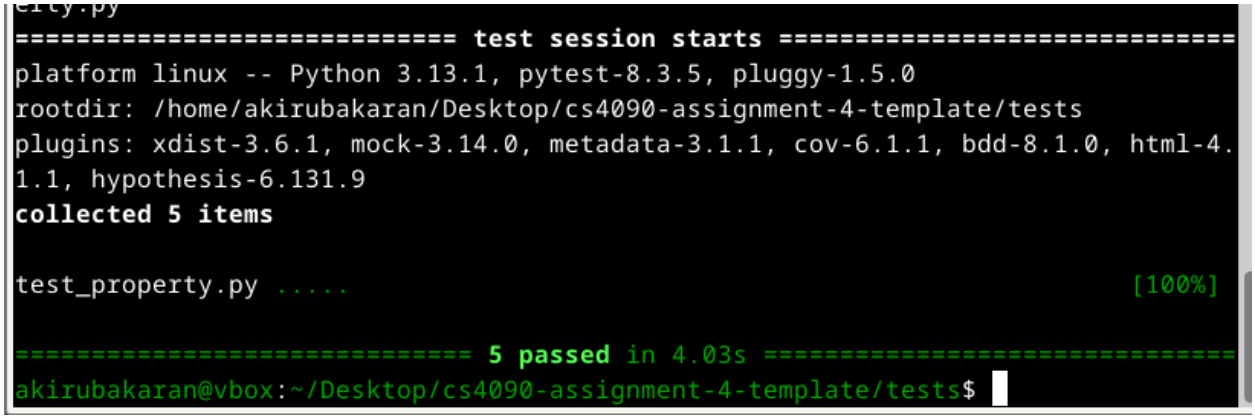
Ensures tasks can contain punctuation or symbols like “!”, “@”, etc. Useful for real-world entries with emoji, shorthand, or hashtags.

All feature files were run from the Streamlit sidebar, providing visual confirmation of each

6. Property-Based Testing

- Property 1: Priority Filter Returns Only Matching Tasks
  Ensures that for any generated list of tasks with random priority values, filtering by "High", "Medium", or "Low" only returns tasks with that priority.
- Property 2: Search Query Appears in Title or Description
  Verifies that a randomly chosen query, when passed to the search function, is present in the title or description of each returned task.
- Property 3: Unique ID is Not Repeated
  Tests that even with a large set of randomly generated task IDs, the generate_unique_id function always returns a new ID not already used.
- Property 4: Completion Filter Accuracy
  Checks that tasks returned by filter_tasks_by_completion all match the completion status requested (True or False), across any randomly generated task list.
- Property 5: JSON Save and Load Consistency
  Confirms that saving a list of randomly generated tasks to a JSON file and reloading it results in exactly the same content.

These properties were tested using the hypothesis library with Streamlit integration, and all tests passed consistently.



Lessons Learned

- Coverage reports showed us which parts of the application were being neglected and pushed us to improve backend test reliability.
- I learned that organizing tests clearly (by purpose and scope) makes debugging faster and report writing easier.
- Handling Python import paths in a multi-folder project setup was more difficult than expected and required systematic structure planning.
- BDD made it easier to communicate and think through user behavior from a non-coding perspective, especially helpful for UI-focused scenarios.